

Design Patterns (Padrões de design)

Sumário

O que é:	2
Classificação:.....	3
Categorias:	3
1. Creational Patterns	3
Factory Method.....	3
Abstract Factory	4
Builder	5
Prototype	6
Singleton	7
2. Structural Patterns	8
Adapter.....	8
Bridge	9
Facade.....	10
Composite	11
Decorator	12
Flyweight	13
Proxy.....	14
3. Behavioural Patterns	15
Chain of responsibility	15
Template Method	16
Exemplos de uso (PHP):	18
Padrão 1:	18
Onde usaria:	18
Como:.....	19
Por quê?.....	19
Exemplo com código.....	20
Padrão 2:	22

Onde usaria:	22
Como:.....	23
Por quê?.....	23
Exemplo com código:.....	24
Padrão 3:	25
Onde usaria:	25
Como:.....	25
Por quê?.....	26
Exemplo com código:.....	26
Padrão 4:	28
Onde usaria:	28
Como:.....	28
Por quê?.....	28
Exemplo com código.....	29
Padrão 5:	31
Onde usaria:	31
Como:.....	31
Por quê?.....	31
Exemplo com código.....	32
Onde me ajudaria no backend?.....	34

O que é:

Os padrões de design são soluções práticas para problemas em comum.

Isso quer dizer que quando algo está dando de errado em um projeto do backend, o Design Pattern precisa intervir.

Classificação:

Pode ser classificado por nível de complexidade do problema e nível de detalhes.

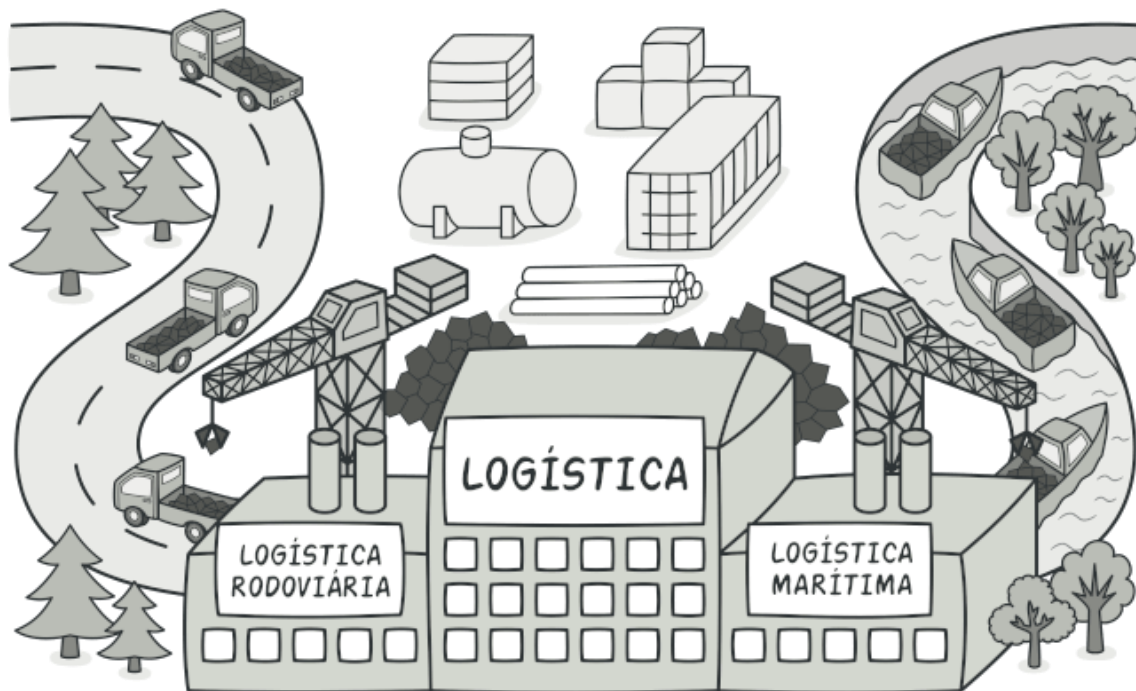
Categorias:

1. Creational Patterns

Creational Patterns fornecem mecanismos de criação de objetos, aumentando assim a flexibilidade e reutilização de código.

Factory Method

O método de fabrica é um padrão POO que ao invés de criar uma classe só para cada produto dela, cria uma superclasse central que as classes produto herdam dela, **impedindo o problema** de quando mudar algo, não ter que mudar completamente tudo, como por exemplo um sistema de transporte, onde a empresa cresce e muda de transporte terrestre para marítimo.



Abstract Factory

O abstract factory é um padrão para criar famílias para que classes e objetos sejam relacionados. **O grande problema** é quando uma classe é usada independente e depois precisa manter o padrão, como por exemplo vários móveis e as variantes desses móveis precisam vir como se fosse um conjunto, assim podendo virar uma família. **A solução** é montar uma interface central, como por exemplo “Cadeira” e

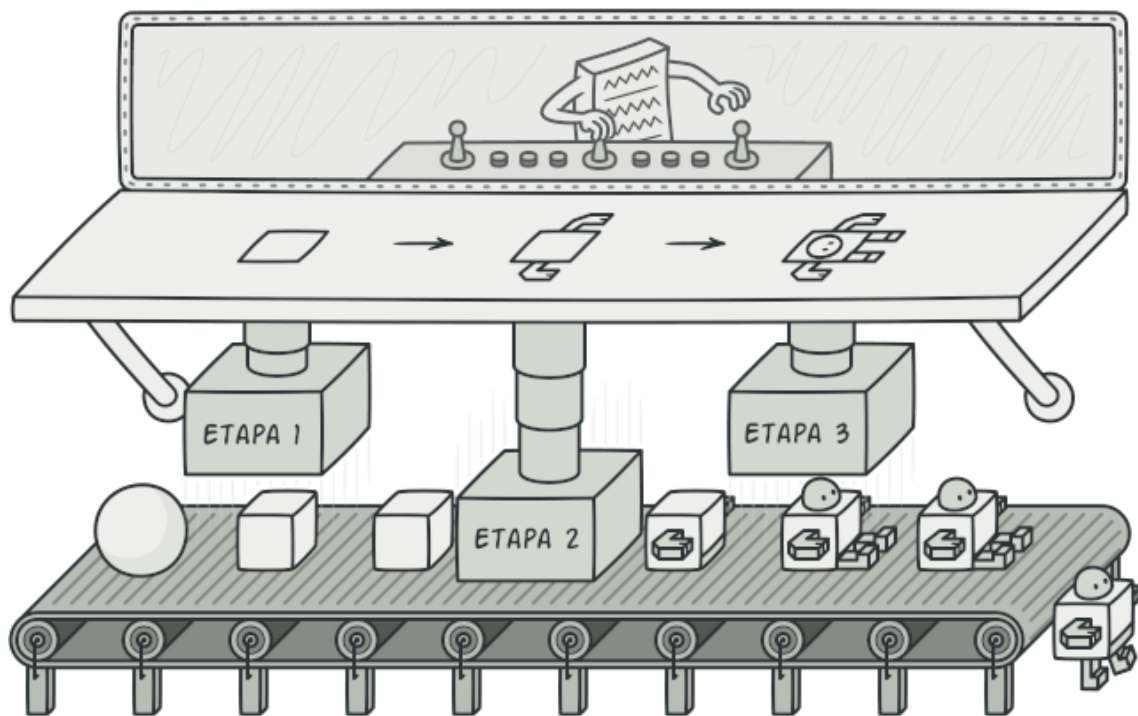
então a subclasse sendo a variante dessa cadeira. Ou declarar a família inteira da variante para colocar na “fabrica de móveis”.



Builder

O builder é um padrão criacional que permite construir objetos complexos passo a passo. **O grande problema** é quando a criação de um objeto exige a passagem de dezenas de parâmetros no construtor, muitos dos quais são opcionais, como por exemplo a construção de uma casa onde você pode escolher ter garagem, piscina ou jardim, gerando um construtor gigante e confuso. **A solução** é separar o código de construção do objeto da sua representação final, utilizando uma classe construtora com métodos encadeados para

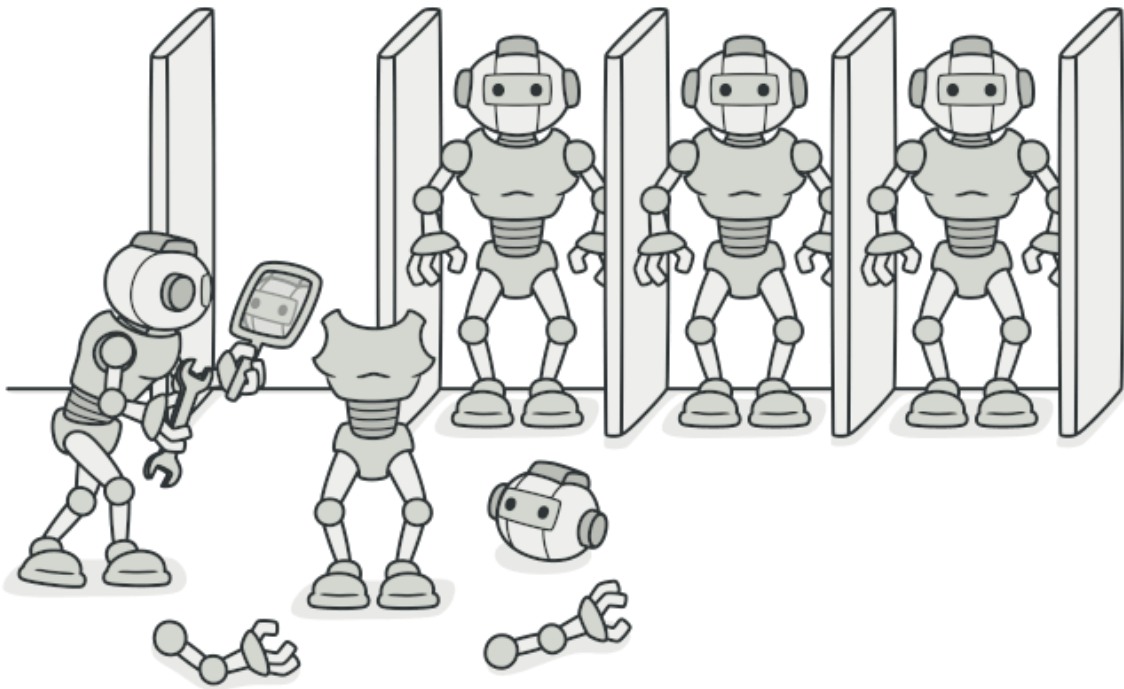
configurar apenas as partes necessárias antes de gerar o objeto final.



Prototype

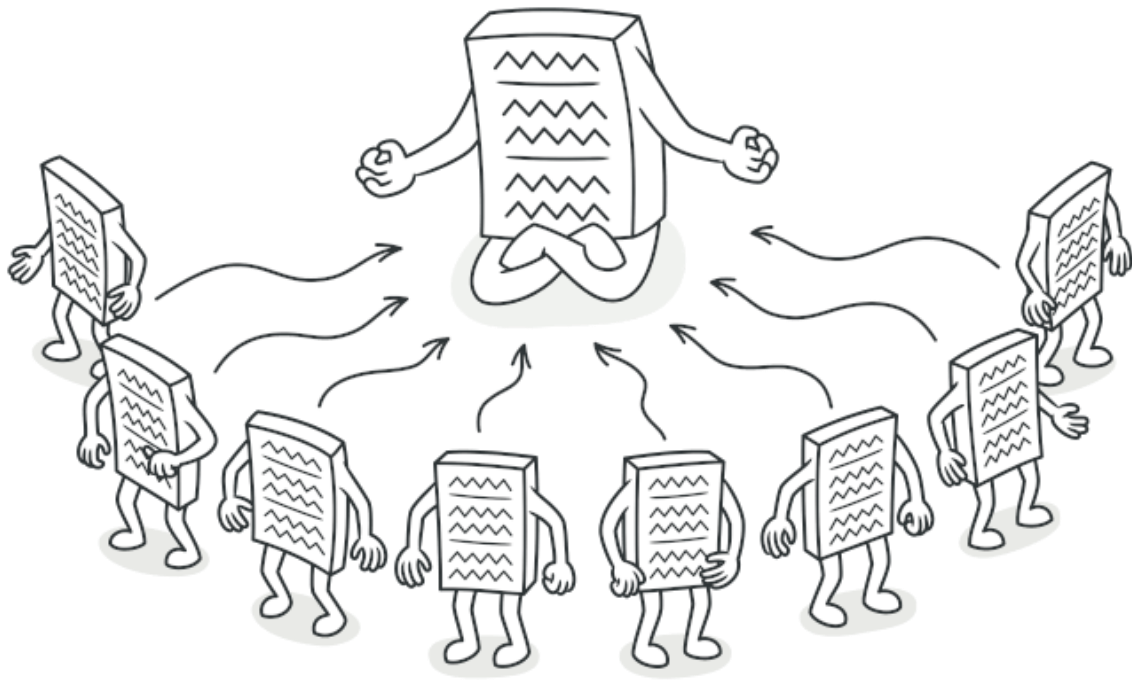
O prototype é um padrão que permite copiar objetos existentes sem tornar seu código dependente de suas classes.

O grande problema é quando você precisa duplicar um objeto que teve um processo de criação muito pesado ou complexo, como por exemplo buscar dados extensos no banco ou realizar cálculos demorados, e instanciar um novo do zero seria ineficiente. **A solução** é delegar a responsabilidade da cópia para o próprio objeto, criando uma interface comum de clonagem para que ele mesmo faça uma cópia de seus atributos internos e retorne um novo objeto idêntico.



Singleton

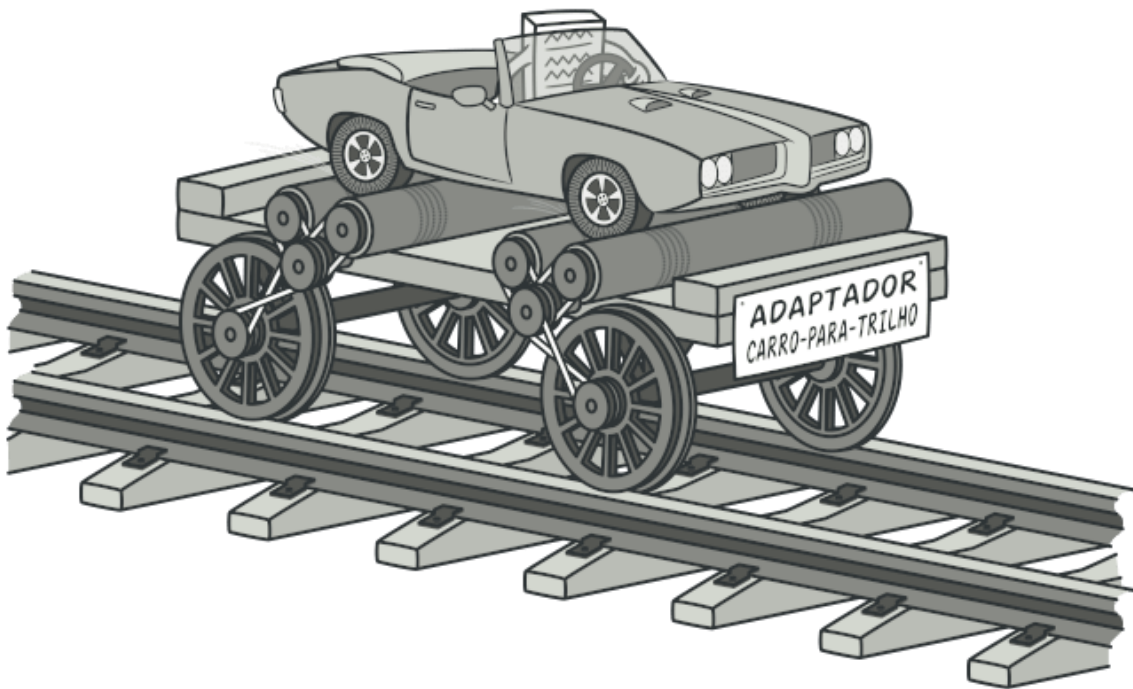
O singleton é um padrão que garante que uma classe tenha apenas uma única instância durante toda a execução da aplicação e fornece um ponto de acesso global a ela. **O grande problema** é quando múltiplos pontos do sistema tentam instanciar repetidamente um recurso compartilhado e crítico, como por exemplo uma conexão com o banco de dados ou um gerenciador de logs, gerando conflitos e desperdício de memória. **A solução** é tornar o construtor da classe privado para impedir novas instanciações externas e criar um método estático que gerencie essa instância única, criando-a apenas na primeira vez que for solicitada e reutilizando-a nas demais.



2. Structural Patterns

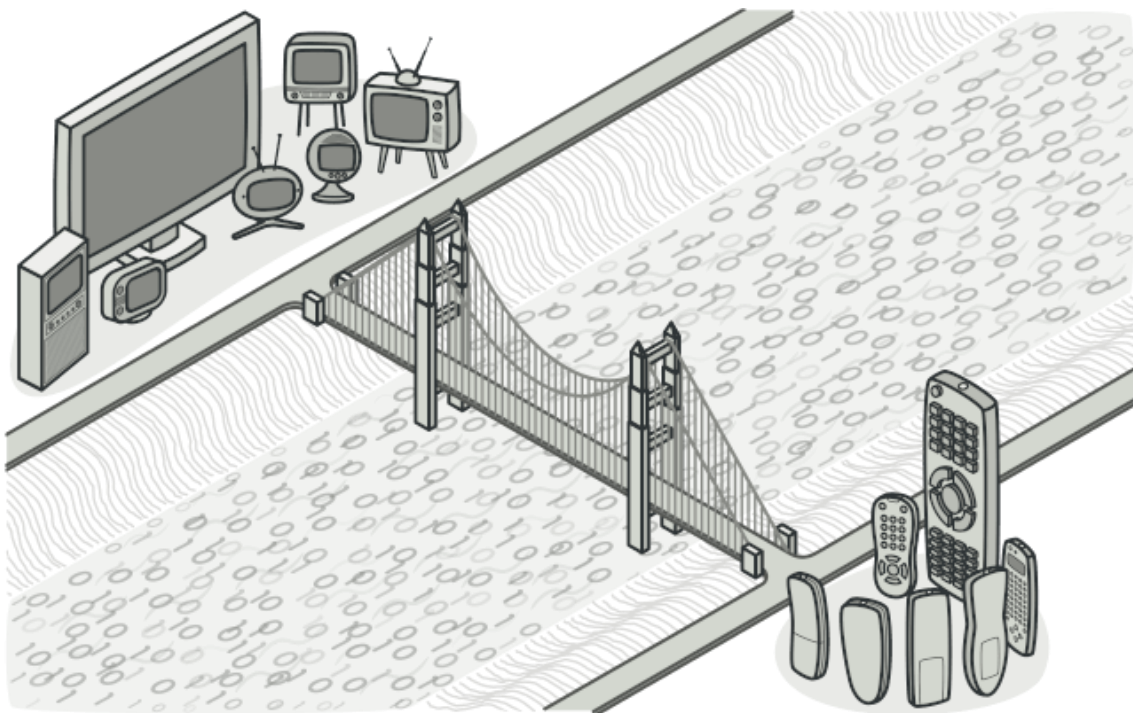
Adapter

O padrão estrutural adapter permite classes se “encaixarem” com outras independente dos dados. Como por exemplo o fato de um banco enviar para sua API dados XML, e você analisar esses dados esperando JSON, é ai que entra o adapter. **A solução** é montar um adapter que permite converter os dados para passar para a outra classe e então enviar os dados que a classe realmente espera.



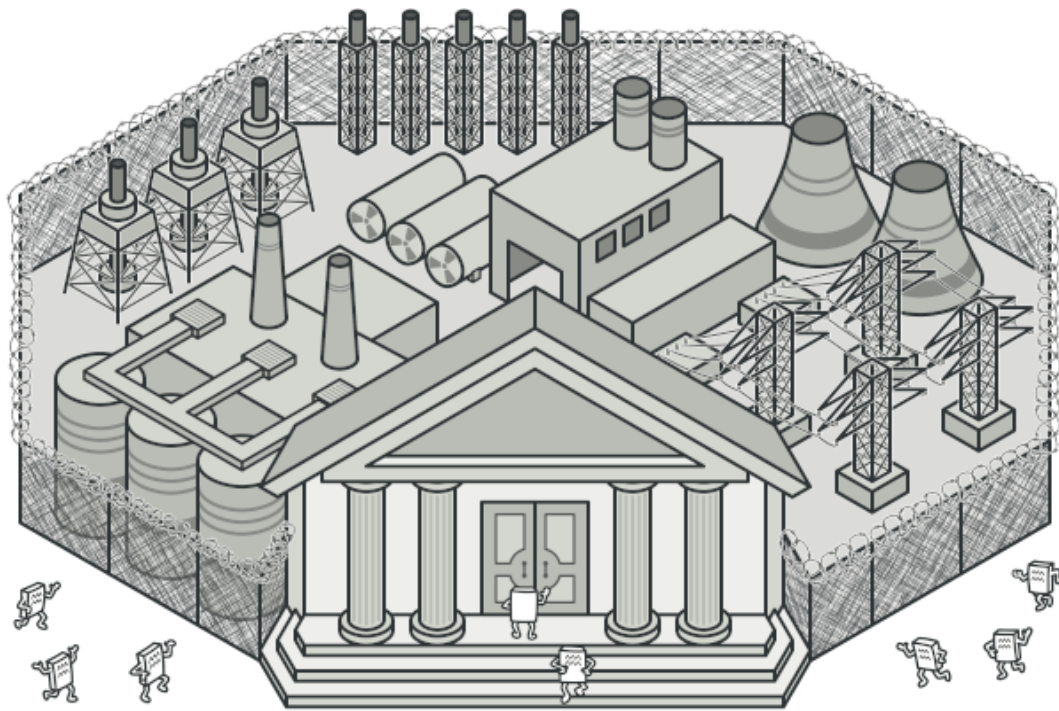
Bridge

O padrão bridge ajuda a criar realmente uma “ponte” entre duas classes pais. **O problema** que o bridge soluciona é quando precisamos ter várias subclasses da mesma classe referenciadas na mesma classe pai, como por exemplo quando temos a classe “Forma” com várias formas geométricas relacionadas a ela, e então as formas podem ter cores, imagina adicionar mais duas subclasse para cada cor das formas geométricas que existem e então se tivéssemos 10 formas, seria 20 classes juntando com cores. **A solução** é realmente fazer uma ponte entre a classe pai forma e a classe pai cor, sendo assim, a forma -> contem -> cor.



Facade

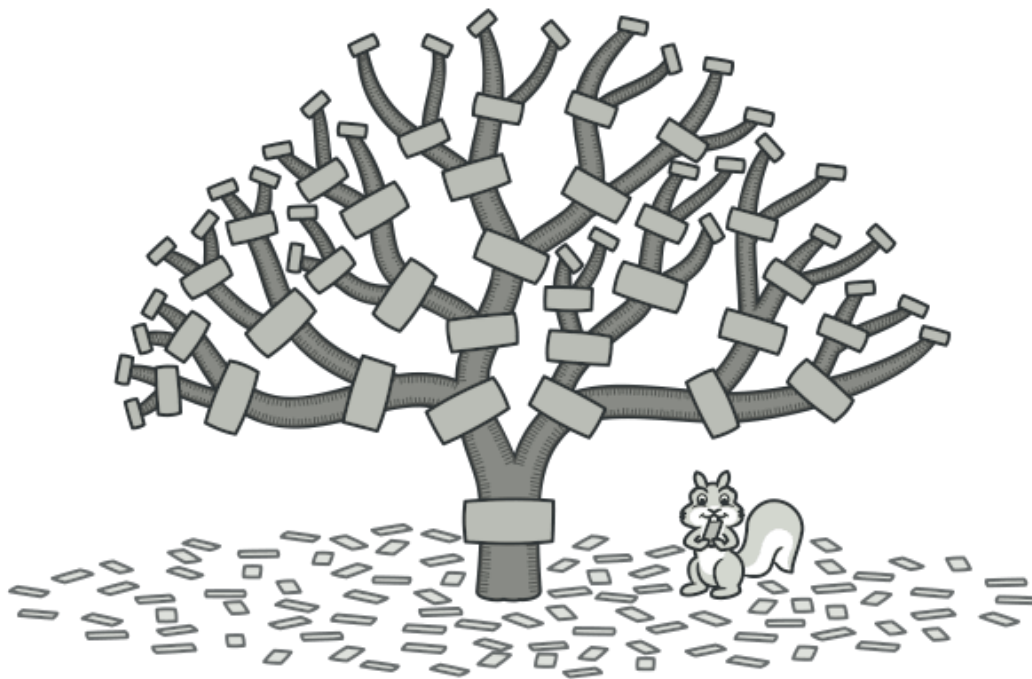
Facade, também conhecido como fachada é um padrão que permite você incluir bibliotecas ou frameworks complexos de forma simples. **Imagine** que você tem uma biblioteca que quer usar, porém ela é super complexa, cheia de funções e métodos, **então você** utiliza o facade para pegar apenas um pedaço dessas funções que você realmente precisa, sem pegar tudo de uma vez. **No mundo real** isso é aquele operador de quando você vai pedir uma pizza, ele que gerencia todas as funcionalidades da pizzeria por trás e te da um retorno simples.



Composite

O composite é um padrão estrutural que permite organizar objetos em estruturas de árvore e trabalhar com essas estruturas como se fossem objetos individuais. **O grande problema** é quando você precisa manipular elementos simples e agrupamentos desses elementos de formas totalmente diferentes no código, como por exemplo calcular o preço total de um pedido que contém produtos avulsos e caixas com múltiplos produtos dentro, gerando loops e verificações condicionais complexas para cada nível. **A solução** é criar uma interface comum tanto para os itens individuais quanto para os recipientes, permitindo que o

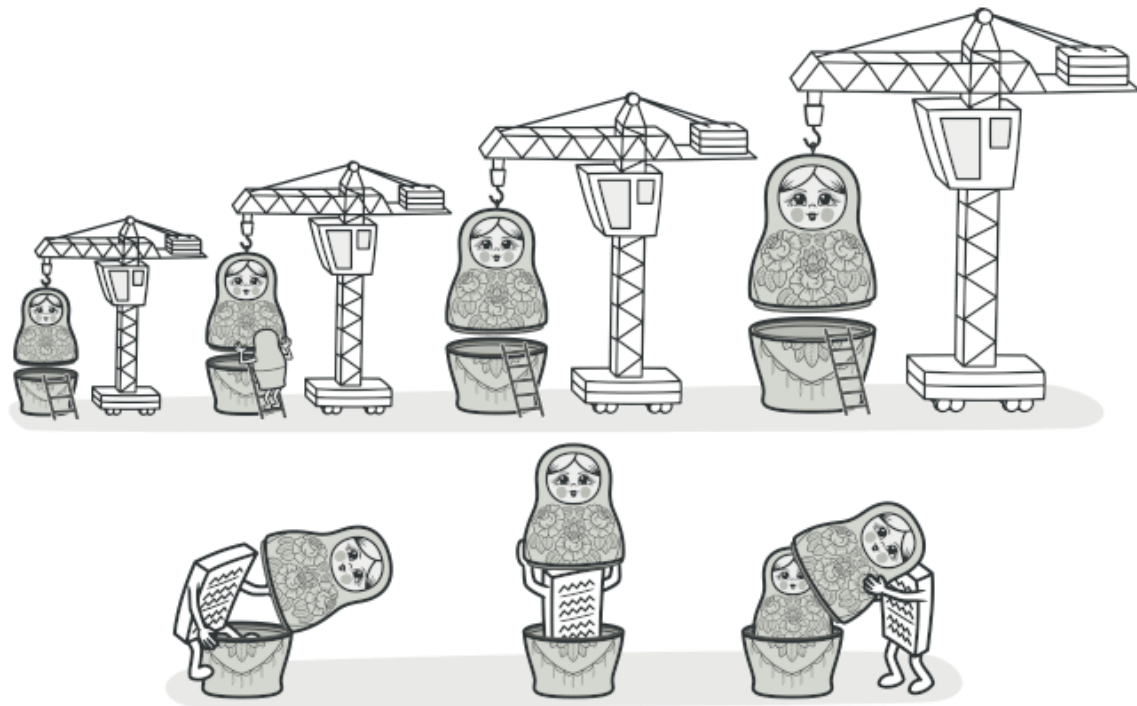
sistema execute a mesma operação em qualquer elemento sem precisar saber se é um item único ou um grupo inteiro.



Decorator

O decorator é um padrão estrutural que permite adicionar novas funcionalidades a um objeto de forma dinâmica sem alterar seu código original. **O grande problema** é quando você tenta usar herança para adicionar comportamentos extras a uma classe e acaba criando um número gigantesco de subclasses para cada combinação de recursos, como por exemplo um sistema de notificação onde o usuário pode escolher receber alertas por e-mail, SMS, WhatsApp ou combinações de todos eles. **A solução** é envolver o objeto original dentro de classes empacotadoras (wrappers) que

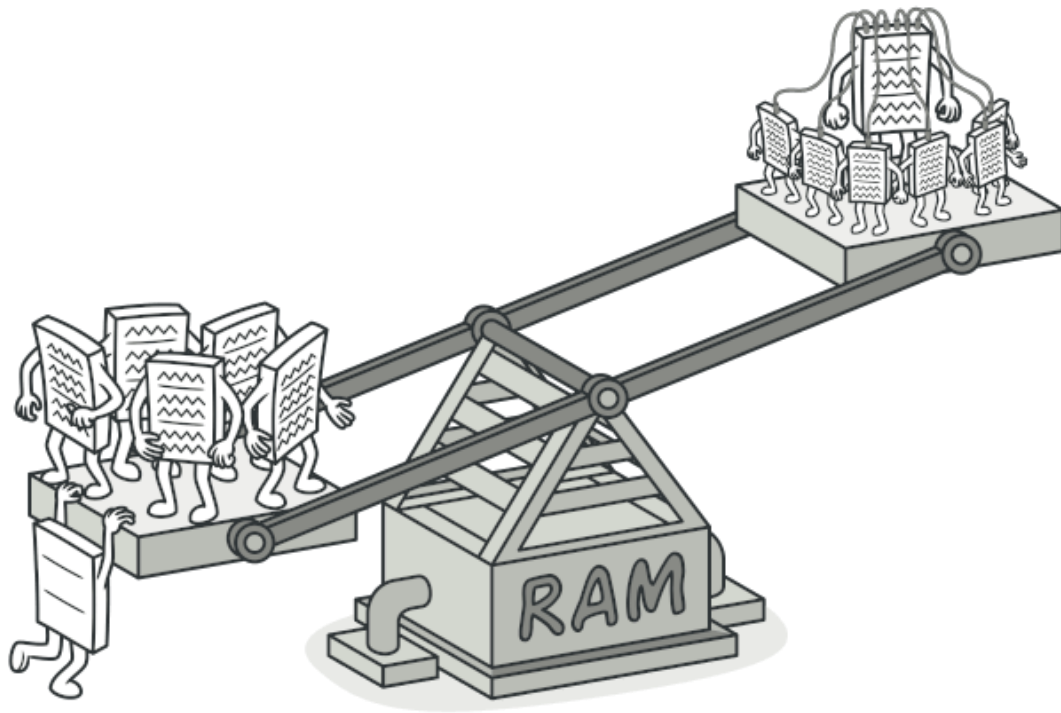
implementam a mesma interface, permitindo empilhar múltiplos comportamentos em tempo de execução.



Flyweight

O flyweight é um padrão estrutural que permite economizar memória RAM compartilhando partes do estado de um objeto entre múltiplos elementos semelhantes. **O grande problema** é quando a aplicação precisa instanciar milhares ou milhões de objetos na memória simultaneamente, como por exemplo carregar centenas de árvores idênticas em um jogo ou milhares de caracteres formatados em um editor de texto, fazendo o sistema esgotar a memória disponível. **A solução** é dividir os dados do objeto entre intrínsecos (dados imutáveis e compartilhados, como a textura e o modelo 3D da árvore) e extrínsecos (dados únicos, como as coordenadas X e Y no

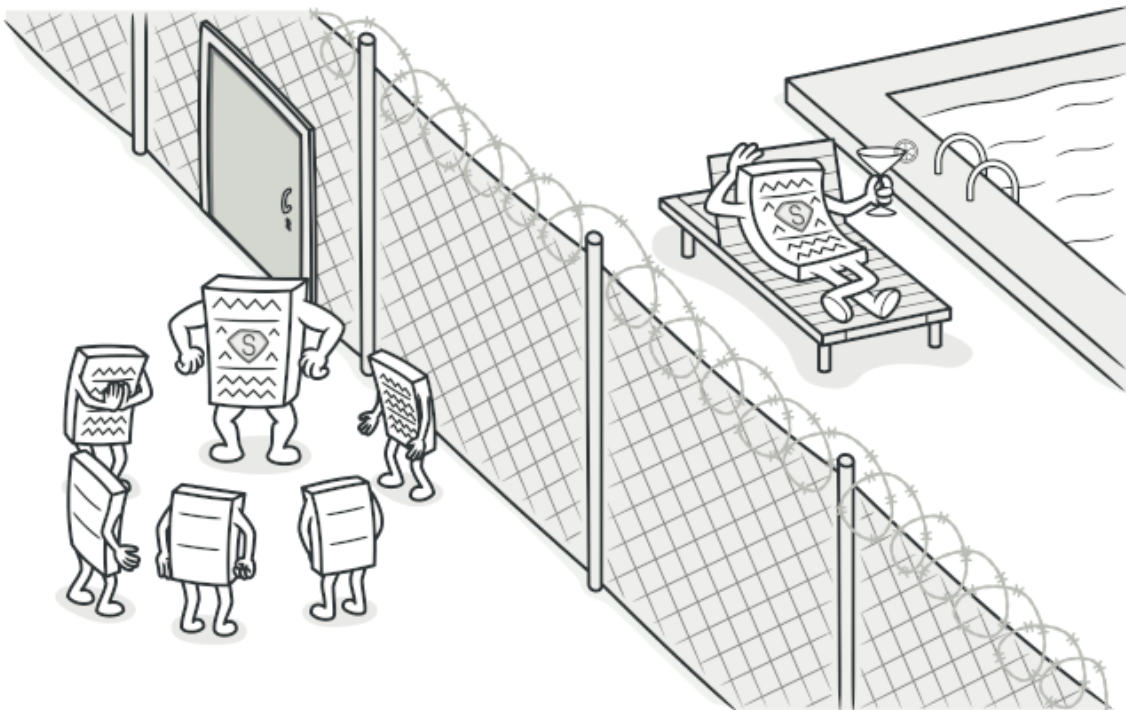
mapa), mantendo na memória apenas uma única instância dos dados compartilhados.



Proxy

O proxy é um padrão estrutural que fornece um substituto ou ponto de intermédio para outro objeto a fim de controlar o acesso a ele. O grande problema é quando você tem um objeto muito pesado ou crítico, como uma conexão de banco de dados ou um serviço de API externa, e precisa adicionar comportamentos extras como cache, controle de permissões de acesso ou inicialização adiada (*lazy loading*) sem alterar o código original desse objeto. A solução é criar uma classe intermediária que implementa a mesma interface do objeto real, funcionando como um "intermediador" que intercepta as chamadas, executa as verificações ou otimizações

necessárias e só repassa a solicitação ao objeto verdadeiro quando realmente apropriado.

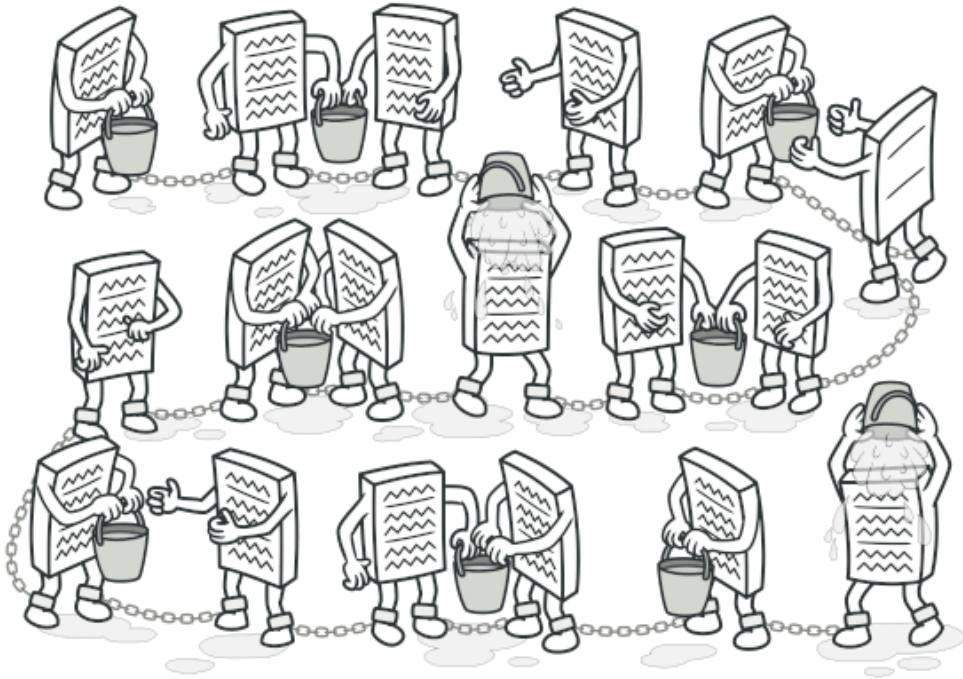


3. Behavioural Patterns

Chain of responsibility

A padrão comportamental cadeia de responsabilidade permite você organizar melhor as checagens/etapas de um processo do seu backend. **O problema** acontece quando você tem muitas checagens em uma só classe/método e quanto mais cresce as checagens, mais incha essa classe/método. **A solução** é fazer um corrente de “handlers”, os carinhas que tem uma só responsabilidade e que quando

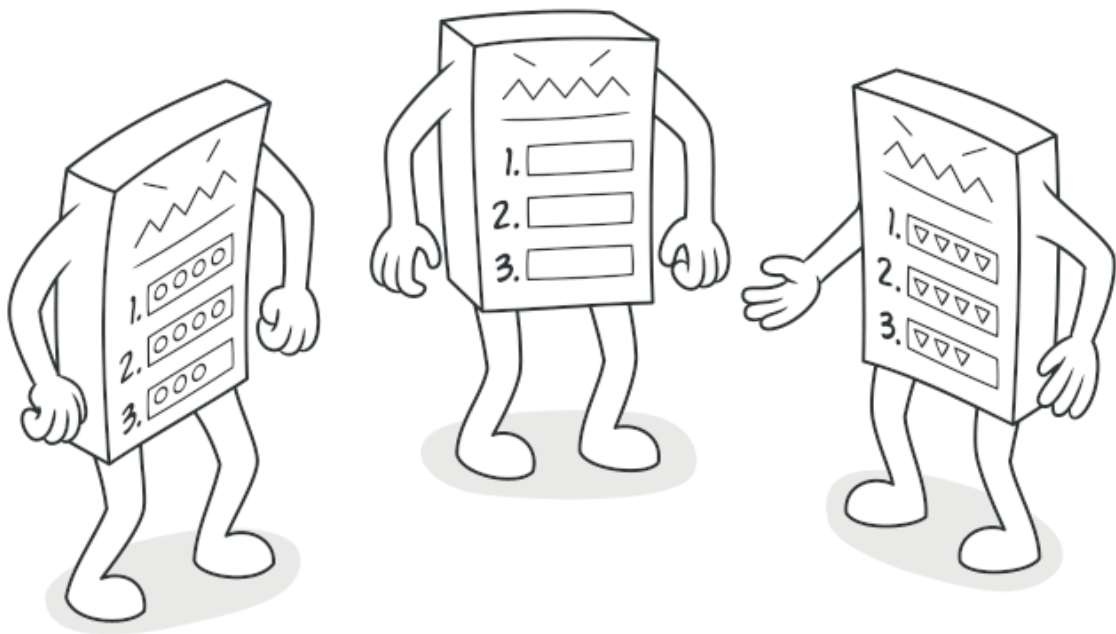
terminam sua responsabilidade passam adiante para o próximo handler da corrente, sendo assim, o sistema fica escalável e toda hora que uma nova checagem precisa ser implementada, um novo handler aparece.



Template Method

O template method é um padrão comportamental que define o esqueleto de um algoritmo em uma superclasse, permitindo que as subclasses sobrescrevam etapas específicas sem alterar a estrutura geral do algoritmo. O grande problema é quando você tem vários fluxos no sistema que seguem exatamente a mesma sequência de passos, mas com pequenas variações em etapas pontuais, como por exemplo leitores de arquivos que precisam abrir o documento, processar os dados (seja de um PDF, CSV ou JSON) e salvar o resultado no banco, levando a muita

duplicação de código para controlar a sequência. A solução é criar um método principal na classe pai que chama todos os passos em ordem fixa e transformar os passos variáveis em métodos abstratos, deixando que cada subclasse implemente apenas a sua lógica específica para aquela etapa.



Entre outros:

- Command
- Iterator
- Mediator
- Memento
- Observer
- State

- Strategy
- Visitor

Exemplos de uso (PHP):

Padrão 1:

Chains Of Responsibility

Onde usaria:

Há diversos contextos que esse padrão poderia ser utilizado, mas um dos contextos mais simples pode ser um sistema de banco. O banco disponibiliza várias checagens do usuário, se ele está logado, se tem saldo para fazer uma transação, se o destinatário existe e outras checagens de segurança. O jeito que eu usaria seria exatamente nesse sistema de banco, onde várias etapas precisam ser checadas antes de uma ação ser realizada de verdade, com o caso de uma transferência/compra.

Como:

Para deixar bem simples, vou dividir o sistema inteiro em um pedaço pequeno só, como a funcionalidade de receber/enviar um pix. O pix quando é enviado gerencia várias credenciais e fatores de segurança para que, quando for transferido não tenha alguém malicioso que consiga interceptar e alterar o valor ou dados sensíveis. Na transferência é necessário verificar o saldo, verificar credibilidade do usuário (se está logado, não é uma conta falsa etc.), validar se o destinatário existe, descontar o valor da conta do remetente e depois acrescentar esse valor na conta do destinatário.

Por quê?

A explicação do porque utilizei esse padrão num sistema de banco é exatamente pelo motivo do que é o Chains of responsibility, nele cita a criação de vários handlers que cada um tem uma “responsabilidade” independente. Isso é extremamente útil se por exemplo o banco adotasse uma nova validação, ao invés de ter que colocar novamente um código dentro de um arquivo inteiro ou classe, dividiria em uma outra classe que passa entre as informações da transferência e a finalização do código, assim pequenas etapas viram uma grande execução.

Exemplo com código

Esse padrão nada mais é que um middleware no PHP. Na pasta middleware no lumen (framework) é onde acontece todas as validações, como por exemplo a verificação se o usuário está logado (auth), se ele tem dinheiro (checkDinheiroRemente) e assim por diante. Segue o código que relata bem a responsabilidade no middleware.

```
<?php

namespace App\Http\Middleware;

use Closure;

class Auth {
    public function handle($request, Closure $next){
        $token = $request->header('Authorization');

        if(empty($token)){
            return response()->json([
                'sucesso' => false,
                'mensagem' => 'Usuário não
autenticado'
            ], 401);
        }

        return $next($request);
    }
}
```

```
}
```

```
<?php

namespace App\Http\Middleware;

use Closure;

class CheckDinheiroRemetente
{
    public function handle($request, Closure $next)
    {
        $token = $request->header('Authorization');
        $usuario = Usuario::where('token', $token)->first(); // Buscar o usuário no banco de dados com o model
        // pelo token guardado na tabela

        $transacao = Transacao::where('usuario_id')->first();
        // Buscar o carrinho do usuário para verificar o valor da transação.

        // Verifica se tem dinheiro

        if ($usuario->dinheiro > $transacao->valor)
        {
            return response()->json([
                'sucesso' => false,
                'mensagem' => 'Usuário sem dinheiro'
            ], 422);
        }
    }
}
```

```
    }  
  
    // Passa para a próxima requisição  
    return $next($request);  
}  
}
```

Fonte: <https://refactoring.guru/pt-br/design-patterns/chain-of-responsibility/php/example#example-1>

Padrão 2:

Facade

Onde usaria:

Sistema de dashboard geral, que mostre indicadores sobre o clima, visitantes em um shopping e preços de produtos.

A lógica seria a seguinte: Entender e relatar no dashboard como os preços e o clima afetam os visitantes em um shopping, com filtros, ordenação, categorias, etc.

A ideia é realmente entender como isso poderia ser resolvido e os problemas os visitantes enfrentam diariamente.

Como:

O clima, visitantes e preços seriam todos uma API externa que disponibiliza informações cruciais para o meu sistema de dashboard. Nessas API's eu nem sempre precisaria utilizar todos os indicadores de clima, por exemplo, então usaria o padrão FACADE com o objetivo de minimizar a implementação de métodos das API's que não serão utilizados, deixando mais simples.

Por quê?

O Padrão Facade ajuda muito nisso quando não preciso obrigatoriamente utilizar todos os métodos externos dentro do meu sistema, deixando a “FACHADA” mais apresentável.

Exemplo com código:

```
<?php

use GuzzleHttp\Client;

// Instancia o cliente do Guzzle (Biblioteca usada
para buscar APIs e links externos para o php)

$client = new Client();

// Busca a API.
$res = $client->request('GET',
    'https://api.clima.com/chuva');

// Aparece na tela o status dessa requisição.
echo $res->getStatusCode();

// Aparece na tela o corpo dessa requisição, como
por exemplo quantos milímetros vai chover em um
sábado qualquer, onde as
// pessoas costumam ir no shopping
echo $res->getBody();
```

Padrão 3:

Bridge

Onde usaria:

Esse padrão é um pouco mais específico, porém pode também ser utilizado em vários contextos, mas o que mais cabível seria o contexto de uma concessionária de veículos esportivos. Imagina só, uma concessionária tem vários carros, cada carro tem várias cores, estilos, designs e muito mais, se eu fosse colocar uma classe herdeira de uma classe pai (Veículo) e depois colocar mais subclasses para cada cor, estilos e designs, isso viraria algo extremamente exponencial e esse padrão não indica fazer isso.

Como:

Eu utilizaria o padrão bridge para criar uma “ponte” entre duas classes pai, como por exemplo a classe pai (Veículo) -> contém (ponte) -> classe ponte(cor/design/estilo), assim toda vez que eu precisasse adicionar um novo estilo a todos os carros, eu apenas mudaria a classe ponte estilo e pronto, todas elas agora têm uma nova cor que pode ser utilizada.

Por quê?

Porque assim evita a exponenciação das subclasses de uma classe pai e vira algo super escalável.

Exemplo com código:

```
<?php

// 1. As cores
interface Cor {
    public function getNome(): string;
}

// Implements aqui porque é para Classe ->
// Interfaces e extends é para Classe -> Classe.
class Vermelho implements Cor {
    public function getNome(): string { return
    "Vermelho"; }
}

// Cria uma nova cor com base na interface de cor e
// retorna cor.
class Azul implements Cor {
    public function getNome(): string { return
    "Azul"; }
```

```
}

// 2. O Veículo (recebe a cor por parâmetro).
class CarroEsportivo {
    private Cor $cor; // Esta propriedade é a
    "Ponte" (Bridge).

    public function __construct(Cor $cor) {
        $this->cor = $cor;
    }

    public function mostrar(): string {
        return "Carro Esportivo " . $this->cor->getNome();
    }
}

// 3. Montando o objeto na prática.
$carroVermelho = new CarroEsportivo(new
Vermelho());
$carroAzul = new CarroEsportivo(new Azul());

echo $carroVermelho->mostrar(); // Carro Esportivo
Vermelho
echo $carroAzul->mostrar();      // Carro Esportivo
Azul
```

Padrão 4:

Factory Method

Onde usaria:

Em um sistema de e-commerce que precisa processar diferentes métodos de pagamento (como Pix, Cartão de Crédito e Boleto).

Como:

O sistema possui uma classe base de checkout que define a regra geral de finalização da compra, mas delega a criação do objeto de pagamento específico para um método (o Factory Method). Quando o cliente escolhe a forma de pagamento na interface, o sistema aciona o criador correspondente que instancia o processador correto sem que o fluxo principal precise saber os detalhes de cada forma de pagamento.

Por quê?

Elimina o acoplamento direto entre o fluxo de checkout e as classes concretas de pagamento. Se no futuro o sistema

precisar aceitar um novo meio de pagamento (como Criptomoedas ou Carteira Digital), basta criar uma nova classe de processador e uma subclasse criadora, mantendo o código do e-commerce intacto e seguro contra bugs

Exemplo com código

```
<?php

// Interface do produto
interface ProcessadorPagamento {
    public function processar(float $valor):
string;
}

// Produtos concretos
class PagamentoPix implements ProcessadorPagamento
{
    public function processar(float $valor): string
    {
        return "Pagamento de R$ {$valor} gerado via
QR Code PIX.";
    }
}

class PagamentoCartao implements
ProcessadorPagamento {
    public function processar(float $valor): string
    {
        return "Pagamento de R$ {$valor} aprovado
no Cartão de Crédito.";
```

```

    }
}

// Classe Criadora Abstrata (contém o Factory Method)
abstract class GatewayCheckout {
    // Factory Method
    abstract public function criarProcessador():
    ProcessadorPagamento;

    public function finalizarPedido(float $valor):
    string {
        $processador = $this->criarProcessador();
        return $processador->processar($valor);
    }
}

// Criadores concretos
class CheckoutPix extends GatewayCheckout {
    public function criarProcessador():
    ProcessadorPagamento {
        return new PagamentoPix();
    }
}

class CheckoutCartao extends GatewayCheckout {
    public function criarProcessador():
    ProcessadorPagamento {
        return new PagamentoCartao();
    }
}

// Uso no sistema
$checkout = new CheckoutPix();

```

```
echo $checkout->finalizarPedido(150.00);
```

Padrão 5:

Abstract Method

Onde usaria:

Em um sistema de personalização ou catálogo de móveis, onde os produtos pertencem a famílias e estilos específicos (por exemplo, estilo Moderno e estilo Clássico).

Como:

Em vez de criar móveis soltos, definimos uma interface para a fábrica abstrata que possui métodos para criar cada item do conjunto (criarCadeira() e criarSofa()). As fábricas concretas (FabricaModerna e FabricaClassica) garantem que, ao selecionar um estilo, todos os objetos gerados pertençam estritamente à mesma linha de design.

Por quê?

Garante a compatibilidade dos objetos criados, impedindo o risco do sistema misturar produtos de famílias diferentes

(como combinar uma cadeira clássica entalhada com um sofá futurista). Além disso, facilita a inclusão de famílias inteiras de novos produtos (como estilo Industrial) sem alterar o código que utiliza esses móveis.

Exemplo com código

```
<?php

// Interfaces dos produtos da família
interface Cadeira {
    public function exhibeEstilo(): string;
}

interface Sofa {
    public function exhibeEstilo(): string;
}

// Produtos concretos da família "Moderna"
class CadeiraModerna implements Cadeira {
    public function exhibeEstilo(): string {
        return "Cadeira de metal minimalista.";
    }
}

class SofaModerno implements Sofa {
    public function exhibeEstilo(): string {
        return "Sofá retilíneo minimalista.";
    }
}

// Produtos concretos da família "Clássica"
```

```
class CadeiraClassica implements Cadeira {
    public function exhibeEstilo(): string {
        return "Cadeira de madeira entalhada.";
    }
}

class SofaClassico implements Sofa {
    public function exhibeEstilo(): string {
        return "Sofá de veludo trabalhado.";
    }
}

// Interface da Fábrica Abstrata
interface FabricaMoveis {
    public function criarCadeira(): Cadeira;
    public function criarSofa(): Sofa;
}

// Fábricas Concretas para cada família
class FabricaModerna implements FabricaMoveis {
    public function criarCadeira(): Cadeira {
return new CadeiraModerna(); }
    public function criarSofa(): Sofa { return new
SofaModerno(); }
}

class FabricaClassica implements FabricaMoveis {
    public function criarCadeira(): Cadeira {
return new CadeiraClassica(); }
    public function criarSofa(): Sofa { return new
SofaClassico(); }
}

// Uso no sistema
```

```
function montarSala(FabricaMoveis $fabrica): void {  
    $cadeira = $fabrica->criarCadeira();  
    $sofa = $fabrica->criarSofa();  
  
    echo $cadeira->exibeEstilo() . "\n";  
    echo $sofa->exibeEstilo() . "\n";  
}  
  
// Cliente escolhe a linha clássica  
montarSala(new FabricaClassica());
```

Onde me ajudaria no backend?

Diante de alguns Design Patterns que pesquisei, entendi e achei mais importantes, o único que realmente me ajudaria seria um que já faço automaticamente no meu backend, que é o caso do “Chains Of Responsibility” como middleware da minha aplicação.

O resto dos padrões não me interessou o uso na competição por enquanto já que tem alguns que demonstra muita complexidade.

Continua sendo algo bom e útil de estudar para a worldskills ou nacional caso for pedido algum desses padrões.

